

Hardened Images Launch Labspace

A Labspace to get started with the launch of the new hardened images in your own private environment.

Introduction

👋 Welcome to the **Hardened Images Launch** lab! During this lab, you will learn to do the following:

- Pull & Run Hardened Images
- Do Multi-stage build with Hardened Images
- Use Compose with Hardened Images

Throughout this lab you will use a repository containing a Restaurant reservation (**dinner**) application consisting of a basic Python app talking to a PostgreSQL database to demonstrate the use of Hardened Images.

Prerequisites

First things to get started, please provide the following registry URLs:

What is your Hardened Images Container Registry prefix?

Update

What is your GitHub Container Registry (ghcr.io) remote repository URL?

Update

What is your DockerHub remote repository URL ?

Update

Alternatively, if no customization are needed, you can set their value to their industry default:

Set default URLs

Configured registry URLs:

- Hardened Images Container registry URL: dhi.io/
 - GitHub Container Registry URL: ghcr.io
 - DockerHub URL: docker.io
-

Pull & Run

Pull the Python image

Pull the hardened Python image from your private registry:

```
docker pull dhi.io/python:3.14-debian13
```

Run the Python image

Run locally this hardened Python image:

```
docker run --rm dhi.io/python:3.14-debian13 python -c 'print("Hello, Hardened Image!")'
```

Test the "No shell" variant

Try to run a shell with this hardened Python image:

```
docker run --rm -it dhi.io/python:3.14-debian13 sh
```

You get this error message:

```
OCI runtime create failed: runc create failed: unable to start container process: error during container init: exec: "sh": executable file not found in $PATH
```

You cannot run any commands because this **runtime** variant image doesn't have a shell, a package manager or any extra system packages. We will illustrate this part in the next section.

Test the access to internal website

Identify a website with your corporate issued TLS certificate:

What is a website only accessible with your corporate issued TLS certificate?

docker.com

Update

```
echo "Q" | openssl s_client -connect docker.com:443 -showcerts 2>/dev/null | openssl x509 -noout -issuer
```

Try to access an internal website from this hardened Python image:

```
docker run --rm dhi.io/python:3.14-debian13 python -c 'import urllib.request;
print(urllib.request.urlopen("https://docker.com", timeout=10).status)'
```

You should see a successful **200** response.

Use DHI

Update the Dockerfile to use an hardened **dev** base image

Change the base image in the **FROM** instruction in the [Dockerfile](#) and save.

```
FROM dhi.io/python:3.14-debian13-dev
```

```
- FROM python:3.14
+ FROM dhi.io/python:3.14-debian13-dev
```

Build the updated image with the hardened **dev** variant:

```
docker build -t dinner:dev .
```

Compare the CVEs between the initial image and the latest hardened **dev** image built locally:

```
docker scout compare \
  --ignore-unchanged \
  --to ghcr.io/mathieu-benoit/dinner:initial \
  dinner:dev
```

	Analyzed Image	Comparison Image
Target	dinner:dev	ghcr.io/mathieu-benoit/dinner:initial
digest	839b85db4441	08cd3eb3bac2
tag	dev	initial
platform	linux/amd64	linux/amd64
provenance		https://github.com/mathieu-benoit/dhi-launch.git#refs/heads/main:project
vulnerabilities	0C 6H 9M 37L 1? -12 -82 -75 -230 -92	12C 88H 84M 267L 93?
size	64 MB (-373 MB)	437 MB
packages	181 (-311)	492

See the number of CVEs, packages and size of the image just got improved:

- CVEs: -491
- Packages: -311
- Size (on disk): -373MB

- Run-as: `root`

Test the "shell" variant

Try to run a shell with this hardened `dev` image built locally:

```
docker run --rm -it dinner:dev sh
```

You can run some commands because this `dev` variant image has a shell, a package manager and extra system packages.

```
whomai  
cat /etc/os-release
```

Exit the opened shell:

```
exit
```

Multi-stage build

Update the Dockerfile to use a runtime base image

Update the [Dockerfile](#) with this content:

```
FROM dhi.io/python:3.14-debian13-dev AS builder
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt --target /app

FROM dhi.io/python:3.14-debian13 AS runtime
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
COPY --from=builder /app /app
COPY . /app
EXPOSE 5000
CMD ["python", "/app/app.py"]
```

```
docker build -t dinner:runtime .
```

Compare the CVEs between the `dev` and the `runtime` hardened images built locally:

```
docker scout compare \
  --ignore-unchanged \
  --to dinner:dev \
  dinner:runtime
```

	Analyzed Image	Comparison Image
Target	dinner:runtime	dinner:dev
digest	13c9caec76bf	839b85db4441
tag	runtime	dev
platform	linux/amd64	linux/amd64
vulnerabilities	0C 6H 8M 22L 1?	0C 6H 9M 37L 1?
size	38 MB (-26 MB)	64 MB
packages	106 (-75)	181

See the number of CVEs, packages and size of the image just got improved:

- CVEs: -16
- Packages: -75
- Size (on disk): -26MB
- Run-as: `root` → `nonroot`

Test the "no shell" variant

Try to run a shell with this hardened `runtime` image built locally:

```
docker run --rm -it dinner:runtime sh
```

You get this error message:

```
OCI runtime create failed: runc create failed: unable to start container process: error during container  
init: exec: "sh": executable file not found in $PATH
```

You cannot run any commands because this `runtime` variant image doesn't have a shell, a package manager or any extra system packages.

Note: Out of scope of this workshop, but instead, you can use the `docker debug` or `kubectl debug` commands to attach a temporary, tool-rich debug container to the running instance.

Compose

Modify the Docker Compose file to use the PostgreSQL image

Change the db image in the [compose.yaml](#) and save.

```
image: dhi.io/postgres:18-debian13
```

```
- image: postgres:18  
+ image: dhi.io/postgres:18-debian13
```

Run the application

Run the application:

```
docker compose up -d --build
```

Check that the containers are running successfully:

```
docker ps
```

Go to the application in the browser: <http://localhost:5001>.

Enter a reservation using the application to confirm that the application is working.

Scan the PostgreSQL image

Compare the CVEs between the initial PostgreSQL image and the hardened PostgreSQL image:

```
docker scout compare \  
  --ignore-unchanged \  
  --to docker.io/postgres:18 \  
  dhi.io/postgres:18-debian13
```

	Analyzed Image	Comparison Image
Target	dhi.io/postgres:18-debian13	postgres:18
digest	ea877f94242a	0c49c0c906cb
tag	18-debian13	latest
platform	linux/amd64	linux/amd64
provenance	https://github.com/docker-hardened-images/definitions 4945910b7a0d0bb66cd3047cd3918762ceb99a47	https://github.com/docker-library/postgres.git 4f9ced003ba58a854656ba150d146243d27ae3ac
vulnerabilities	0C 0H 0M 2L	8C 38H 40M 74L 6?
size	128 MB (-34 MB)	162 MB
packages	131 (-74)	205

See the number of CVEs, packages and size of the image just got improved:

- CVEs: -164
- Packages: -74
- Size (on disk): 34MB
- Run-as: `root` → `postgres`

Conclusion

You've completed the **Hardened Images Launch** lab!

✅ You now know how to:

- Pull & Run Hardened Images
- Do Multi-stage build with Hardened Images
- Use Compose with Hardened Images

Here are the benefits illustrated with the Python container image:

	Default Python	DHI Python	
CVEs	12C 88H 84M 267L 93?	6H 8M 22L 1?	-507
Size (on disk)	437 MB	38 MB	-399 MB
Number of packages	492	106	-386
Run-as	root	nonroot	
Package manager	Yes	No	
Shell	Yes	No	

And here are the benefits illustrated with the PostgreSQL container image:

	Default PostgreSQL	DHI PostgreSQL	
CVEs	8C 38H 40M 74L 6?	2L	-164
Size (on disk)	162 MB	128 MB	-34 MB
Number of packages	205	131	-74
Run-as	root	postgres	
Package manager	Yes	No	
Shell	Yes	Yes	

🎉 Well done!

Resources

- [Multi-stage builds](#)
- [Base image hardening](#)

- [Troubleshoot hardened images](#)